MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Rotaru Anastasia
Group FAF-242

# Report

Laboratory Work No. 5
Operating Systems
Topic: The File System with Your Own Hands (Linux)

Checked by:
Reițman Patricia, asist. univ.

Chisinau – 2026

## Contents


**1 INTRODUCTION AND GOAL** **3**

1.1 Introduction . . . . . 3

1.2 Lab Objective . . . . . 3

**2 THE TREE AND FILE TYPES** **4**

2.1 Command Execution and Raw Output . . . . . 4

2.2 Analysis and Observations . . . . . 4

**3 INODES AND LINKS** **5**

3.1 Command Execution and Raw Output . . . . . 5

3.2 Analysis and Observations . . . . . 5

**4 PERMISSIONS AND SPECIAL BITS** **7**

4.1 Command Execution and Raw Output . . . . . 7

4.2 Analysis and Observations . . . . . 7

**5 DISKS AND THE KERNEL AS FILES** **9**

5.1 Command Execution and Raw Output . . . . . 9

5.2 Disk Layout Diagram . . . . . 10

5.3 Analysis and Observations . . . . . 10

**6 STRETCH GOALS** **12**

6.1 Stretch Goal A: Magic Numbers . . . . . 12

6.2 Stretch Goal C: Deletion of an Active Open File . . . . . 12

**CONCLUSION** **13**

**REFERENCES** **14**

# 1 INTRODUCTION AND GOAL

## 1.1 Introduction

Operating systems provide high-level abstractions to manage persistent secondary storage, device communication, and running process states. Under the UNIX and Linux design philosophy, these resources are unified under a hierarchical virtual filesystem tree. Rather than interacting directly with block addresses on disk hardware, user applications access files, directories, character interfaces, storage devices, and kernel tables through standardized POSIX system call abstractions.

## 1.2 Lab Objective

The primary objective of this laboratory work is to investigate the foundational mechanisms of the Linux filesystem using direct command-line utilities and kernel inspections. The scope covers:

- the filesystem tree and file type classification from file metadata;
- inode structures, directory entry namespaces, hard links, and symbolic links;
- file size semantics versus allocated physical disk space (sparse files);
- Discretionary Access Control (DAC) permissions, directory traversal rules, umask operations, and special permission bits (Setuid, Sticky bit);
- disk partitioning, mount points, and pseudo-filesystems exposing kernel internal state (`/proc`).

# 2 THE TREE AND FILE TYPES

## 2.1 Command Execution and Raw Output

The system directory root and core entry points were inspected using `ls`, `file`, and `du`:

```
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ls -ld /etc /etc/passwd /bin /dev/null /
    tmp
lrwxrwxrwx  1 root root    7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-  1 root root 1, 3 Oct 25 21:41 /dev/null
drwxr-xr-x  1 root root 4096 Oct 22 08:58 /etc
-rw-r--r--  1 root root 1115 Sep 10 12:29 /etc/passwd
drwxr-xrwt+ 4 root root 4096 Oct 25 21:42 /tmp

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ls -l /dev | grep '^b'
brw-rw---- 1 root disk 8,  0 Oct 25 21:41 sda
brw-rw---- 1 root disk 8,  1 Oct 25 21:41 sda1
brw-rw---- 1 root disk 8, 14 Oct 25 21:41 sda14
brw-rw---- 1 root disk 8, 15 Oct 25 21:41 sda15
brw-rw---- 1 root disk 8, 16 Oct 25 21:41 sdb
brw-rw---- 1 root disk 8, 17 Oct 25 21:41 sdb1
brw-rw---- 1 root disk 8, 32 Oct 25 21:41 sdc
brw-rw---- 1 root disk 8, 33 Oct 25 21:41 sdc1

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ du -sh /var/log 2>/dev/null ; du -sh /
    usr 2>/dev/null
828K    /var/log
7.0G    /usr
```

## 2.2 Analysis and Observations

a) **File types from the initial character of `ls -ld`:**

- `/bin`: `l` → Symbolic link.
- `/dev/null`: `c` → Character device.
- `/etc`: `d` → Directory.
- `/etc/passwd`: `-` → Regular file.
- `/tmp`: `d` → Directory with sticky bit protection.

b) **Nature of `/bin`:** On modern Linux distributions, `/bin` is a symbolic link pointing to `/usr/bin`. This follows the UsrMerge architecture, consolidating all core binaries under `/usr`.

c) **Device classification in `/dev`:**

- Character device (`c`): `/dev/null` processes data as an unbuffered stream of single bytes.
- Block device (`b`): Devices like `/dev/sda`, `/dev/sda1`, `/dev/sdb1`, and `/dev/sdc1` transfer data through the kernel page cache in fixed-size blocks.

d) **Disk usage comparison:** `/usr` occupies **7.0 GB**, whereas `/var/log` occupies **828 KB**. The `/usr` hierarchy is significantly larger because it houses core userland programs, shared libraries, and documentation, whereas `/var/log` only stores rotating system logs.

# 3 INODES AND LINKS

## 3.1 Command Execution and Raw Output

Hard links, symbolic links, directory structure links, and file allocation properties were evaluated:

```
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ echo hello > notes.txt
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 0,45    Inode: 1443212     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/codespace)   Gid: ( 1000/codespace)

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ln notes.txt hard.txt
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ls -li notes.txt hard.txt
1443212 -rw-r--r-- 2 codespace codespace 6 Oct 25 21:49 hard.txt
1443212 -rw-r--r-- 2 codespace codespace 6 Oct 25 21:49 notes.txt

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ rm notes.txt
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ cat hard.txt ; ls -li hard.txt
hello
1443212 -rw-r--r-- 1 codespace codespace 6 Oct 25 21:49 hard.txt

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ln -s hard.txt soft.txt
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ls -li hard.txt soft.txt
1443212 -rw-r--r-- 1 codespace codespace 6 Oct 25 21:49 hard.txt
1443213 lrwxrwxrwx 1 codespace codespace 8 Oct 25 21:49 soft.txt -> hard.txt

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ rm hard.txt
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 codespace codespace 8 Oct 25 21:49 soft.txt -> hard.txt

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ mkdir d ; ls -ld d
drwxr-xr-x 2 codespace codespace 4096 Oct 25 21:49 d

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 codespace codespace 4096 Oct 25 21:49 d

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ echo x > one.txt ; ls -l one.txt ; du -h
     one.txt
-rw-r--r-- 1 codespace codespace 2 Oct 25 21:49 one.txt
4.0K    one.txt

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ truncate -s 1G big.img ; ls -lh big.img
    ; du -h big.img
-rw-r--r-- 1 codespace codespace 1.0G Oct 25 21:49 big.img
0       big.img
```

## 3.2 Analysis and Observations

a) **Inode numbers and link reference counters:** The field `Inode: 1443212` specifies the unique inode index. The `Links: 1` parameter defines how many directory namespace entries point to that inode.

b) **Hard link data persistence:** Creating a hard link introduces a second directory entry sharing the identical inode (`1443212`). The command `rm notes.txt` invokes the `unlink()` system call,

decrementing the link count from 2 to 1. Because the link count remains non-zero, the physical data blocks remain intact and accessible via `hard.txt`.

c) **Symbolic link behavior:** A symlink creates a new inode (`1443213`) holding the text string path `"hard.txt"`. When `hard.txt` is unlinked, reading `soft.txt` fails with `No such file or directory` because target path resolution fails. The size of `soft.txt` is exactly **8 bytes** because the ASCII string `"hard.txt"` consists of 8 bytes.

d) **Directory link counting:** A new directory `d` has **2 links**: one from its parent directory entry pointing to it, and its internal `.` (dot) entry. Adding subdirectories `d/x` and `d/y` increases the count to **4** because each subdirectory contains a `..` (dot-dot) entry pointing back to the parent directory.

e) **Physical block allocation versus logical size:**

- `one.txt` contains 2 bytes, but the filesystem allocates disk space in minimum block units of 4096 bytes (4.0 KB).
- `big.img` is a *sparse file* created via `truncate`. The kernel stores a logical size of 1.0 GB in the inode metadata without allocating physical disk extents until data is written, resulting in 0 bytes reported by `du`.

# 4 PERMISSIONS AND SPECIAL BITS

## 4.1 Command Execution and Raw Output

Standard permissions, directory traversal rules, umask calculation, and elevated bits were tested:

```
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ echo secret > s.txt
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 codespace codespace 7 Oct 25 21:49 s.txt

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ chmod u+r s.txt ; cat s.txt
secret

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ umask ; touch new.txt ; mkdir newdir ;
    ls -ld new.txt newdir
0022
-rw-r--r-- 1 codespace codespace    0 Oct 25 21:49 new.txt
drwxr-xr-x 2 codespace codespace 4096 Oct 25 21:49 newdir

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ mkdir box ; echo hi > box/f
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
drwxr-xrwt+ 4 root root 4096 Oct 25 21:42 /tmp

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ cat /etc/shadow
cat: /etc/shadow: Permission denied

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ ls -l /etc/shadow
-rw-r----- 1 root shadow 603 Sep 10 12:29 /etc/shadow
```

## 4.2 Analysis and Observations

a) **Octal permission representations:**

- `600` (`-rw-------`): Read and write for owner only. *Use:* Private configuration files or SSH keys.
- `644` (`-rw-r--r--`): Read and write for owner, read-only for group and others. *Use:* Public text documents or source code files.
- `755` (`-rwxr-xr-x`): Read, write, execute for owner; read and execute for group/others. *Use:* Executable binaries and shared scripts.
- `700` (`-rwx------`): Read, write, execute for owner only. *Use:* Private user bin directories or secure script folders.

b) **Ownership versus `chmod 000`:** Linux evaluates file permission bits based on the matching credential tier. Ownership merely selects which bit triplet (owner, group, or others) applies to the user; it does not grant automatic read/write privileges. Setting permissions to `000` creates an explicit block for all classes, including the owner. Only the `root` user bypasses these checks.

c) **Umask mechanics:** The active umask is `0022`.

$$\text{For files (default base 666)}: \quad 666_8 \wedge \neg(0022_8) = 644_8 \quad (\texttt{-rw-r--r--}), \text{ matching } \texttt{new.txt}.$$
$$\text{For directories (default base 777)}: \quad 777_8 \wedge \neg(0022_8) = 755_8 \quad (\texttt{drwxr-xr-x}), \text{ matching } \texttt{newdir}.$$

d) **Directory permissions (`r` vs. `x`):**

- Read permission (`r`) allows listing the contents of a directory (e.g., `ls`).
- Execute permission (`x`) allows path traversal through a directory to access files inside it or query metadata.
- In the experiment: With `u=r,go=` `box`, you could list files (`ls box`), but could not read `box/f` because you lacked execute permission to enter the directory path. With `u=x,go=` `box`, you could not list the directory contents (`ls` failed), but you *could* read `box/f` if you knew its exact name because the execute bit permitted traversal.

e) **Special permission bits:**

- The `s` (SetUID) bit on `/usr/bin/passwd` means the program runs with the privileges of its owner (**root**) rather than the invoking user. `passwd` needs this so regular users can modify restricted system files like `/etc/shadow`.
- The `t` (sticky) bit on `/tmp` ensures that files within a world-writable directory can only be deleted, renamed, or modified by their respective owners or root, preventing users from deleting each other's temporary files.

# 5 DISKS AND THE KERNEL AS FILES

## 5.1 Command Execution and Raw Output

Storage partitioning, mount tables, and virtual kernel interfaces were inspected:

```
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ lsblk
NAME    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0     7:0    0    74M  1 loop
loop1     7:1    0 115.3M  1 loop
loop2     7:2    0  50.1M  1 loop
loop3     7:3    0   257G  0 loop
loop4     7:4    0    32G  0 loop /var/lib/containerd
                                  /home/vscode/.minikube
                                  /workspaces/.codespaces/.persistedshare
                                  /var/lib/docker
                                  /etc/hosts
                                  /etc/hostname
                                  /etc/resolv.conf
                                  /workspaces
loop5     7:5    0   500M  0 loop
sda       8:0    0    30G  0 disk
   sda1       8:1    0  29.9G  0 part /workspaces/.codespaces/shared
                                     /.codespaces/bin
                                     /vscode
                                     /usr/sbin/docker-init
   sda14      8:14   0     4M  0 part
   sda15      8:15   0   106M  0 part
sdb       8:16   0    45G  0 disk
   sdb1       8:17   0    45G  0 part /tmp
sdc       8:32   0   512G  0 disk
   sdc1       8:33   0   512G  0 part
sr0      11:0    1   628K  1 rom

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ mount | grep "␣/␣"
overlay on / type overlay (rw,relatime,lowerdir=/var/lib/docker/overlay2/l/
    KSEMGVJEKSYD4C56ITBY44F7XJ:/var/lib/docker/overlay2/l/ZJQ5245WW4Q7A7CYK3P2SZYIMD:/var/lib/
    docker/overlay2/l/QAXJCL2DNKHR5CCD6DQC2LMWOI:/var/lib/docker/overlay2/l/
    O76VGHRRHDS6BJMZGYYOYU6J6E:/var/lib/docker/overlay2/l/COQXJ2XREH624H74URK4CVGXUC:/var/lib/
    docker/overlay2/l/ML2O6SDGWCC5TJX2EFYBST3N5B:/var/lib/docker/overlay2/l/
    ELYOP56KWROTLPAQSMLU6IKEBB:/var/lib/docker/overlay2/l/FPW77L3ZCBZNRRW7GTLVML4HMO:/var/lib/
    docker/overlay2/l/F6D5ZMBBNNCIETGPM3RUZK4F7W:/var/lib/docker/overlay2/l/2
    QDNJ3WTTHUB2EGTBWIKYUGXE3:/var/lib/docker/overlay2/l/TFSHNOLTZT6J4G5HVHJPS2O4A2:/var/lib/
    docker/overlay2/l/IESA3SYTHMX3WJRCPIKGLGQFBW:/var/lib/docker/overlay2/l/
    WEUKK4JTM33YFTNVNQBHXREXAU:/var/lib/docker/overlay2/l/3QVGGB5BKGQBNZVQY6V75HDJ74:/var/lib/
    docker/overlay2/l/EXPM3Y4PNKW5AIB3KU3S2DJY77:/var/lib/docker/overlay2/l/
    ER5HPNE5VXZYUFVCFRMM2IF3A3:/var/lib/docker/overlay2/l/T5TITQE2I2XTXY37YAVPDNABKG:/var/lib/
    docker/overlay2/l/6GL5DTSKK6GHFESR4E423ZVT7L:/var/lib/docker/overlay2/l/
    ZIQ7VJCSZ53KPD5JEM3CMIV2R7:/var/lib/docker/overlay2/l/YMAHWBML5WSQKSJYAA23EMPQMT:/var/lib/
    docker/overlay2/l/ZZMPRI3FNYA3IQLTOYXLCLMUIN:/var/lib/docker/overlay2/l/
    R5A7XXBOEGRC2JPFWKXWDEQ7CT:/var/lib/docker/overlay2/l/4NDQD2K5BPLULZZEIJQZGF5FFQ:/var/lib/
    docker/overlay2/l/MTPUTL7KJXDXPTSPR2ZMJST5M5:/var/lib/docker/overlay2/l/3
    KXALDXVJM5UU7HDAYRK6QUJVW:/var/lib/docker/overlay2/l/Z3GVQMJJNJZQJGDAN53PLTOIE3:/var/lib/
    docker/overlay2/l/YTOZCMPAF2PQB5EGJLYTZHQ4GD:/var/lib/docker/overlay2/l/
    JPRFBSZMTSALEBNBX77F5VMYPE:/var/lib/docker/overlay2/l/WTS3UUQI4A6HROUZPMI6KCWJU3:/var/lib/
    docker/overlay2/l/JBQX3Q2O3IFFEVEFDAVHS4MTFG:/var/lib/docker/overlay2/l/
    CAZGISGVDPUKDCJ4USIA5BQC3W:/var/lib/docker/overlay2/l/OJPTCDH5VWMIMLQ7Q5JPJBRWON,upperdir=/
    var/lib/docker/overlay2/1f3cfda4c0954e57fb15198ce26bdd23dcde74e6a99890a40b157e3b9e909afe/
    diff,workdir=/var/lib/docker/overlay2/1
    f3cfda4c0954e57fb15198ce26bdd23dcde74e6a99890a40b157e3b9e909afe/work,nouserxattr)

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ df -h /
```

```
Filesystem      Size  Used Avail Use% Mounted on
overlay          32G   12G   19G  38% /

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ grep -c processor /proc/cpuinfo
2

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ head -3 /proc/meminfo
MemTotal:        8131872 kB
MemFree:          186508 kB
MemAvailable:    6000536 kB

@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   5765
Ngid:   0
```

### 5.2 Disk Layout Diagram

Below is the structural diagram illustrating how the physical block storage devices and loopback mounts map to system mount points:

```
+-------------------------------------------------------------------------+
|                            Disk Architecture                            |
+-------------------------------------------------------------------------+
| [/dev/sda] 30 GB Physical Disk                                          |
|            /dev/sda1  (29.9G) --> Mounted: /vscode, /.codespaces/bin     |
|            /dev/sda14 (4M)    --> BIOS Boot Partition                   |
|            /dev/sda15 (106M)  --> EFI System Partition                  |
+-------------------------------------------------------------------------+
| [/dev/sdb] 45 GB Physical Disk                                          |
|            /dev/sdb1  (45G)   --> Mounted: /tmp                         |
+-------------------------------------------------------------------------+
| [/dev/sdc] 512 GB Physical Disk                                         |
|            /dev/sdc1  (512G)  --> Storage Partition                     |
+-------------------------------------------------------------------------+
| [Virtual Overlay Filesystem] 32 GB                                      |
|            overlay              --> Mounted: / (Root Filesystem, 38% Used)  |
+-------------------------------------------------------------------------+
```

### 5.3 Analysis and Observations

a) **Storage architecture:** The system presents primary disks `sda` (30G), `sdb` (45G), and `sdc` (512G). The root directory (`/`) is mounted on an **`overlay`** filesystem layer of size 32G.

b) **Filesystem capacity:** The root partition is currently **38% full** (12G used out of 32G, with 19G available).

c) **Hardware metrics exposed in `/proc`:**

- Processor cores: **2 processors** (`/proc/cpuinfo`).
- Total system RAM: **8,131,872 kB** (≈ 7.76 GB).

d) **Behavior of `/proc/self/status`:** The virtual link `/proc/self` resolves dynamically to the PID of the calling process accessing the filesystem entry. Because `head` opened the status file, the kernel reported the metadata belonging to `head`.

# 6 STRETCH GOALS

## 6.1 Stretch Goal A: Magic Numbers

The file signature header of a gzip-compressed archive was examined:

```
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ gzip -c /etc/passwd > notes2.txt ; file
    notes2.txt ; xxd notes2.txt | head -1
notes2.txt: gzip compressed data, was "passwd", last modified: Thu Sep 10 12:29:43 2026, from
    Unix, original size modulo 2^32 1115
00000000: 1f8b 0808 b7a2 a26a 0003 7061 7373 7764  .......j..passwd
```

**Analysis:** The initial two hexadecimal bytes `1f 8b` represent the magic number identifying gzip streams according to RFC 1952. The `file` utility inspects these leading bytes against its magic database, allowing accurate file classification regardless of file extension.

## 6.2 Stretch Goal C: Deletion of an Active Open File

The filesystem reaction to unlinking a file with an active descriptor was tested using `exec 3<&-`:

```
@anastasiarotaru     /workspaces/OS-III/lab5 (main) $ echo ceva > bigfile; exec 3< bigfile; rm
     bigfile; ls -l /proc/$$/fd; exec 3<&-
lr-x------ 1 codespace codespace 64 Oct 25 21:49 3 -> /home/codespace/os-lab5/bigfile (deleted
    )
```

**Analysis:** File descriptor 3 retains an open link to the deleted inode. Although `rm` removed the directory namespace entry, the inode and its allocated space remain reserved until descriptor 3 is closed with `exec 3<&-`.

# CONCLUSION

This laboratory work provided a direct, practical inspection of the Linux filesystem architecture and its underlying POSIX mechanics. Performing these experiments on a live Linux system demonstrated how userland commands map directly to kernel filesystem abstractions.

The most important insights gained from this laboratory work are:

- **Separation of names and data:** Directory entries are merely names mapped to inode pointers; file metadata and data blocks exist independently of the directory entries that reference them.
- **Link mechanics:** Hard links share identical inode structures directly, while symbolic links are separate files storing destination string paths.
- **Deferred inode reclamation:** Unlinking a file only removes its directory entry; actual data blocks are only returned to the free list when the link count reaches zero and all processes close open file descriptors.
- **Permission semantics:** Directory permissions require both read (`r`) to list file names and execute (`x`) to traverse into subdirectories and resolve inodes.
- **Kernel exposure via pseudo-filesystems:** Virtual filesystems such as `/proc` demonstrate the UNIX concept of presenting live hardware configurations and process states through regular file read operations.

Overall, this practical work bridged theoretical knowledge of filesystem design with actual operating system behavior, reinforcing how file storage, security checks, and process resources are managed by the Linux kernel.

## REFERENCES

[1] Robert Love, *Linux System Programming: Talking Directly to the Kernel and C Library*, O'Reilly Media.

[2] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press.

[3] Linux Kernel Documentation, *The /proc Filesystem*, https://docs.kernel.org/filesystems/proc.html

[4] POSIX IEEE Std 1003.1-2017, *Standard for Information Technology—Portable Operating System Interface (POSIX)*, File Systems and Permissions.